TP 1 : Tests Unitaires & Coverage

pytest · SQLite :memory: · Coverage.py · Mocking

Objectif du TP

À l'issue de ce TP, vous serez capable de : écrire une suite de tests unitaires complète avec pytest, utiliser une base SQLite en mémoire comme fixture de test, mocker des dépendances externes (Redis, services tiers), et mesurer la couverture de code avec Coverage.py en atteignant au minimum 80%.

Stack	Python 3.11 · pytest · pytest-cov · pytest-mock · SQLAlchemy · SQLite · faker
Livrable	Suite tests/unit/ + rapport coverage HTML + réponses aux questions

Contexte : Projet ShopFlow

ShopFlow est une API e-commerce développée avec FastAPI et SQLite. Elle gère un catalogue de produits, un système de panier et des commandes. Dans ce TP, vous allez écrire les tests unitaires de la couche services (la logique métier) en isolation complète de la base de données et des services externes.

Structure du projet ShopFlow (rappel)

- app/models.py — Modèles SQLAlchemy : Product, Cart, Order, Coupon
- app/services/pricing.py — calcul_prix_ttc(), appliquer_coupon(), calculer_total()
- app/services/stock.py — verifier_stock(), reserver_stock(), liberer_stock()
- app/services/cart.py — ajouter_au_panier(), vider_panier(), calculer_sous_total()
- tests/unit/ — Vos tests unitaires (à créer dans ce TP)
- tests/conftest.py — Fixtures SQLite partagées

Installation de l'environnement (rappel TP0)

```
# 1. Cloner le dépôt ShopFlow (ou récupérer l'archive du formateur)
git clone https://github.com/VOTRE_PSEUDO/shopflow.git
cd shopflow

# 2. Créer et activer l'environnement virtuel
python -m venv venv
source venv/bin/activate          # Linux/Mac
venv\Scripts\activate            # Windows

# 3. Installer les dépendances
pip install -r requirements.txt

# 4. Vérifier l'installation
pytest --version                  # doit afficher pytest 7.x ou 8.x
coverage --version                # doit afficher Coverage.py 7.x
```

Partie 1 : Configuration de pytest et Fixtures SQLite

1.1 Structure des tests

Créez la structure suivante dans le dossier `tests/` :

```
tests/
├── __init__.py
├── conftest.py          # fixtures SQLite partagées
└── unit/
    ├── __init__.py
    ├── test_pricing.py  # tests de pricing.py
    ├── test_stock.py    # tests de stock.py
    └── test_cart.py     # tests de cart.py
```

1.2 Fichier conftest.py : Fixture SQLite en mémoire

Le fichier `conftest.py` centralise les fixtures partagées entre tous les tests. La fixture principale crée une base SQLite en mémoire fraîche à chaque module de test.

```
# tests/conftest.py
# tests/conftest.py
"""
Fichier de configuration pytest - Fixtures partagées entre tous les tests.

Les fixtures définies ici sont automatiquement disponibles dans tous les
fichiers de test sans avoir besoin de les importer.

Hiérarchie des fixtures :
    db_engine → db_session → product_sample / coupon_sample
"""

import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool
from app.database import Base
from app.models import Product, Cart, CartItem, Order, Coupon


# FIXTURES BASE DE DONNÉES

@pytest.fixture(scope="function")
def db_engine():
    """
    Crée un moteur SQLite en mémoire (RAM) pour les tests.

    scope="function" : une nouvelle BDD est créée pour CHAQUE test.
    Cela garantit que les tests sont totalement isolés, les données
    insérées par test_A ne sont jamais visibles par test_B.

    sqlite:///:memory: = BDD en RAM (pas de fichier sur le disque)
    → rapide, isolée, détruite automatiquement après chaque test.

    StaticPool : oblige SQLite à utiliser une seule connexion partagée.
    Obligatoire avec :memory: sinon chaque thread voit une BDD différente.
    """
    engine = create_engine(
```

```
        "sqlite:///:memory:",
        connect_args={"check_same_thread": False},
        poolclass=StaticPool,   # une seule connexion partagée (obligatoire
pour :memory:)
    )

    # Crée toutes les tables définies dans app/models.py
    # (Product, Cart, CartItem, Order, OrderItem, Coupon)
    Base.metadata.create_all(engine)

    yield engine   # le test s'exécute ici

    # Teardown automatique : supprime toutes les tables après le test
    Base.metadata.drop_all(engine)


@pytest.fixture(scope="function")
def db_session(db_engine):
    """
    Fournit une session SQLAlchemy fraîche pour chaque test.

    La session est l'objet de travail SQLAlchemy, c'est via elle
    qu'on fait les INSERT, SELECT, UPDATE, DELETE.

    scope="function" : une nouvelle session pour chaque test individuel.

    Le rollback() final annule toutes les écritures du test :
    même si on oublie de nettoyer les données dans le test,
    la prochaine session repart d'une BDD propre.

    Schéma d'exécution pour chaque test :
        1. Ouvrir la session
        2. Exécuter le test (yield)
        3. Rollback, annule tous les INSERT/UPDATE/DELETE du test
        4. Fermer la session
    """
    Session = sessionmaker(bind=db_engine)
    session = Session()

    yield session   # le test reçoit cet objet session

    # Teardown : annule toutes les modifications du test
    session.rollback()
    session.close()


# FIXTURES DE DONNÉES : objets prêts à l'emploi dans les tests

@pytest.fixture
def product_sample(db_session):
    """
    Crée et insère un produit exemple dans la BDD de test.

    Utilisé dans les tests qui ont besoin d'un produit existant,
    par exemple pour tester reserver_stock() ou ajouter_au_panier().

    Données :
        name  = 'Laptop Pro'
        price = 999.99 € HT
        stock = 10 unités disponibles

    Utilisation dans un test :
        def test_mon_test(product_sample):
            assert product_sample.stock == 10
    """
    p = Product(
```

```
        name="Laptop Pro",
        price=999.99,    # prix HT en euros
        stock=10         # 10 unités disponibles
    )
    db_session.add(p)
    db_session.commit()        # INSERT en base
    db_session.refresh(p)  # recharge l'objet pour récupérer l'id auto-généré
    return p


@pytest.fixture
def coupon_sample(db_session):
    """
    Crée et insère un coupon de réduction exemple dans la BDD de test.

    Utilisé dans les tests qui ont besoin d'un coupon valide,
    par exemple pour tester appliquer_coupon() ou calculer_total().

    Données :
        code      = 'PROMO20'  → identifiant unique du coupon
        reduction = 20.0       → 20% de réduction
        actif     = True       → coupon utilisable

    Exemple d'effet :
        100€ TTC avec PROMO20 → 100 × (1 - 0.20) = 80€

    Utilisation dans un test :
        def test_mon_test(coupon_sample):
            assert coupon_sample.reduction == 20.0
    """
    c = Coupon(
        code="PROMO20",
        reduction=20.0,    # 20% de réduction
        actif=True         # coupon actif = utilisable
    )
    db_session.add(c)
    db_session.commit()    # INSERT en base
    return c
```

1.3 Fichier pytest.ini

Créez le fichier `pytest.ini` à la racine du projet pour configurer les options par défaut :

```
# pytest.ini
[pytest]
addopts = -v --cov=app --cov-report=html --cov-report=term-missing
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*

# Markers personnalisés pour filtrer les tests
markers =
    unit: tests unitaires isolés
    slow: tests lents (> 1s)
    pricing: tests du module pricing
```

Q1 : Fixtures & Configuration

1.1. Créez les fichiers conftest.py et pytest.ini tels que décrits ci-dessus. Lancez pytest --collect-only et copiez la sortie.

1.2. Expliquez la différence entre scope='module' et scope='function' pour une fixture pytest.

1.3. Quel est le rôle du session.rollback() dans la fixture db_session ? Que se passerait-il sans lui ?

1.4. Ajoutez le fichier .coveragerc à la racine pour exclure les migrations et conftest.py du rapport. Montrez son contenu (dejà fait TP0).

Partie 2 : Tests du Module Pricing

Le module `app/services/pricing.py` contient la logique de calcul des prix. C'est le cœur métier de ShopFlow — il doit être testé exhaustivement.

2.1 Code source à tester

Voici le fichier `app/services/pricing.py` que vous allez tester :

```
# app/services/pricing.py
from app.models import Product, Coupon
from typing import Optional

TVA_RATE = 0.20   # Taux de TVA France

def calcul_prix_ttc(prix_ht: float) -> float:
    """Calcule le prix TTC à partir du prix HT."""
    if prix_ht < 0:
        raise ValueError(f'Prix HT invalide : {prix_ht}')
    return round(prix_ht * (1 + TVA_RATE), 2)

def appliquer_coupon(prix: float, coupon: Coupon) -> float:
    """Applique un coupon de réduction sur un prix."""
    if not coupon.actif:
        raise ValueError('Coupon inactif')
    if not 0 < coupon.reduction <= 100:
        raise ValueError(f'Réduction invalide : {coupon.reduction}')
    return round(prix * (1 - coupon.reduction / 100), 2)

def calculer_total(produits: list, coupon: Optional[Coupon] = None) -> float:
    """Calcule le total TTC d'une liste de produits avec coupon optionnel."""
    if not produits:
        return 0.0
    total_ht = sum(p.price * q for p, q in produits)
    total_ttc = calcul_prix_ttc(total_ht)
    if coupon:
        total_ttc = appliquer_coupon(total_ttc, coupon)
    return total_ttc
```

2.2 Écriture des tests test_pricing.py

Créez le fichier `tests/unit/test_pricing.py` et rédigez les tests suivants. Chaque test doit être indépendant et ne tester qu'une seule chose.

```
# tests/unit/test_pricing.py
import pytest
from app.services.pricing import calcul_prix_ttc, appliquer_coupon,
calculer_total
from app.models import Coupon

# ── TESTS calcul_prix_ttc ──────────────────────────────────────
```

```python
@pytest.mark.unit
class TestCalculPrixTtc:
    def test_prix_normal(self):
        """Prix HT 100€ → TTC 120€ avec TVA 20%."""
        assert calcul_prix_ttc(100.0) == 120.0

    def test_prix_zero(self):
        assert calcul_prix_ttc(0.0) == 0.0

    def test_arrondi_deux_decimales(self):
        assert calcul_prix_ttc(10.0) == 12.0   # pas 12.000000001

    def test_prix_negatif_leve_exception(self):
        with pytest.raises(ValueError, match='invalide'):
            calcul_prix_ttc(-5.0)

    @pytest.mark.parametrize('ht,ttc', [
        (50.0, 60.0),
        (199.99, 239.99),
        (0.01, 0.01),
    ])
    def test_parametrise(self, ht, ttc):
        assert calcul_prix_ttc(ht) == ttc

# ── TESTS appliquer_coupon ──────────────────────────────────────
class TestAppliquerCoupon:
    def test_reduction_20_pourcent(self, coupon_sample):
        result = appliquer_coupon(100.0, coupon_sample)
        assert result == 80.0

    def test_coupon_inactif_leve_exception(self, db_session):
        coupon_inactif = Coupon(code='OLD', reduction=10.0, actif=False)
        with pytest.raises(ValueError, match='inactif'):
            appliquer_coupon(100.0, coupon_inactif)

    def test_reduction_invalide(self, coupon_sample):
        coupon_invalide = Coupon(code='BAD', reduction=150.0, actif=True)
        with pytest.raises(ValueError):
            appliquer_coupon(100.0, coupon_invalide)
```

2.3 Paramétrisation avancée

La paramétrisation permet de tester plusieurs jeux de données avec un seul test. Ajoutez ces tests paramétrés à votre fichier `test_pricing.py` :

```python
# Ajoutez à la suite de test_pricing.py
@pytest.mark.parametrize('reduction,prix_initial,prix_attendu', [
    (10, 100.0, 90.0),     # -10%
    (50, 200.0, 100.0),    # -50%
    (100, 50.0, 0.0),      # -100% = gratuit
    (1, 100.0, 99.0),      # -1% minimal
])
```

```
def test_coupon_reductions_diverses(reduction, prix_initial, prix_attendu,
db_session):
    coupon = Coupon(code=f'TEST{reduction}', reduction=float(reduction),
actif=True)
    assert appliquer_coupon(prix_initial, coupon) == prix_attendu
```

Q2 : Tests Module Pricing

2.1. Implémentez tous les tests de TestCalculPrixTtc et TestAppliquerCoupon. Lancez pytest tests/unit/test_pricing.py -v et copiez la sortie.

2.2. Ajoutez les tests paramétrés pour appliquer_coupon. Vérifiez que pytest génère bien 4 cas de test distincts dans la sortie.

2.3. Écrivez le test test_calculer_total_avec_coupon() qui crée 2 produits (50€ et 30€ HT), calcule le total TTC avec PROMO20, et vérifie le résultat.

Partie 3 : Tests du Module Stock & Mocking Redis

Le module `app/services/stock.py` gère la disponibilité des produits. Il interagit avec Redis pour mettre en cache les niveaux de stock. Dans les tests unitaires, Redis doit être mocké (le remplacer par un faux objet pendant les tests, pour ne pas avoir besoin de Redis réellement démarré, il ne doit jamais être réellement appelé).

3.1 Code source stock.py (dejà fourni TP0)

```
# app/services/stock.py
import redis
from app.models import Product
from app.cache import redis_client   # à mocker dans les tests

def verifier_stock(product: Product, quantite: int) -> bool:
    """Vérifie si le stock est suffisant pour la quantité demandée."""
    return product.stock >= quantite

def reserver_stock(product: Product, quantite: int, session) -> Product:
    """Réserve du stock et invalide le cache Redis."""
    if not verifier_stock(product, quantite):
        raise ValueError(f'Stock insuffisant : {product.stock}
disponible(s)')
    product.stock -= quantite
    session.commit()
    redis_client.delete(f'product:{product.id}:stock')  # invalide cache
    return product

def liberer_stock(product: Product, quantite: int, session) -> Product:
    """Libère du stock (annulation commande) et met à jour le cache."""
    product.stock += quantite
    session.commit()
    redis_client.set(f'product:{product.id}:stock', product.stock)
    return product
```

3.2 Tests avec mocking Redis

Créez `tests/unit/test_stock.py`. La clé est de mocker `app.cache.redis_client` pour éviter tout appel réseau réel vers Redis :

```
# tests/unit/test_stock.py
# tests/unit/test_stock.py
"""
Tests unitaires du module app/services/stock.py

Couvre :
  - verifier_stock()  → vérifie si le stock est suffisant
  - reserver_stock()  → décrémente le stock + invalide cache Redis
  - liberer_stock()   → incrémente le stock + met à jour cache Redis
```

```
Point clé : Redis est MOCKÉ dans tous les tests qui appellent
reserver_stock() ou liberer_stock() car ces fonctions appellent
redis_client.delete() ou redis_client.set().
Sans mock → ConnectionRefusedError car Redis n'est pas démarré.
"""

import pytest
from app.services.stock import verifier_stock, reserver_stock, liberer_stock
from app.models import Product

REDIS_MOCK_PATH = 'app.services.stock.redis_client'



# TESTS verifier_stock()
# Fonction pure : lit juste product.stock et retourne True/False.
# Pas besoin de mock Redis, aucun appel réseau.

@pytest.mark.unit
class TestVerifierStock:

    def test_stock_suffisant(self, product_sample):
        """
        Cas nominal : stock=10, demande=5 → True.
        product_sample est fourni par conftest.py (Laptop Pro, stock=10).
        5 <= 10 donc le stock est suffisant.
        """
        assert verifier_stock(product_sample, 5) is True

    def test_stock_insuffisant(self, product_sample):
        """
        Cas d'erreur : stock=10, demande=999 → False.
        999 > 10 donc le stock est insuffisant.
        """
        assert verifier_stock(product_sample, 999) is False

    def test_stock_exactement_disponible(self, product_sample):
        """
        Cas limite : demander exactement le stock disponible → True.
        stock=10, demande=10 → 10 <= 10 → True.
        Important : tester la frontière (ni trop ni trop peu).
        """
        assert verifier_stock(product_sample, 10) is True

    def test_quantite_zero_leve_exception(self, product_sample):
        """
        Quantité = 0 est invalide → ValueError.
        On ne peut pas commander 0 article.
        """
```

```python
        with pytest.raises(ValueError):
            verifier_stock(product_sample, 0)

    def test_quantite_negative_leve_exception(self, product_sample):
        """
        Quantité négative est invalide → ValueError.
        Une quantité doit toujours être > 0.
        """
        with pytest.raises(ValueError):
            verifier_stock(product_sample, -1)



# TESTS reserver_stock()
#
# Cette fonction fait 3 choses :
#   1. Vérifie que le stock est suffisant
#   2. Décrémente product.stock en BDD (session.commit)
#   3. Invalide le cache Redis (redis_client.delete)
#
# → Redis DOIT être mocké pour éviter ConnectionRefusedError
# → On vérifie ensuite que delete() a bien été appelé


class TestReserverStock:

    def test_reservation_reussie(self, product_sample, db_session, mocker):
        """
        Cas nominal : réservation de 3 unités sur stock=10.

        Étapes du test :
            1. Mocker Redis → remplace redis_client par un faux objet
            2. Appeler reserver_stock()
            3. Vérifier que le stock a diminué de 3
            4. Vérifier que Redis.delete() a bien été appelé
               (le cache doit être invalidé après une modification de stock)
        """
        # Étape 1 : mocker Redis
        # mock_redis est un faux objet qui accepte tous les appels
        # sans contacter le vrai Redis
        mock_redis = mocker.patch(REDIS_MOCK_PATH)

        # Mémoriser le stock avant la réservation
        stock_avant = product_sample.stock   # = 10

        # Étape 2 : appeler la fonction à tester
        updated = reserver_stock(product_sample, 3, db_session)

        # Étape 3 : vérifier que le stock a bien diminué
```

```python
    # 10 - 3 = 7 unités restantes
    assert updated.stock == stock_avant - 3

    # Étape 4 : vérifier que Redis.delete() a été appelé exactement 1 fois
    # Si delete() n'est pas appelé → le cache garde l'ancien stock
    # → les clients verraient un stock incorrect
    mock_redis.delete.assert_called_once()

def test_reservation_verifie_cle_redis(self, product_sample, db_session, mocker):
    """
    Vérifie que Redis.delete() est appelé avec la BONNE clé.
    La clé doit identifier précisément le produit modifié.
    Format attendu : "product:{id}:stock"
    """
    mock_redis = mocker.patch(REDIS_MOCK_PATH)

    reserver_stock(product_sample, 1, db_session)

    # Vérifier la clé exacte passée à delete()
    expected_key = f"product:{product_sample.id}:stock"
    mock_redis.delete.assert_called_once_with(expected_key)

def test_stock_insuffisant_leve_exception(self, product_sample, db_session, mocker):
    """
    Stock insuffisant → ValueError avec message 'insuffisant'.
    stock=10, demande=999 → impossible → exception levée.

    Redis est quand même mocké car la fonction appelle
    verifier_stock() avant d'accéder à Redis.
    """
    mocker.patch(REDIS_MOCK_PATH)

    with pytest.raises(ValueError, match='insuffisant'):
        reserver_stock(product_sample, 999, db_session)

def test_stock_insuffisant_ne_modifie_pas_bdd(self, product_sample, db_session, mocker):
    """
    Si le stock est insuffisant, la BDD ne doit PAS être modifiée.
    Le stock doit rester à 10 après l'échec de la réservation.
    """
    mocker.patch(REDIS_MOCK_PATH)
    stock_avant = product_sample.stock   # = 10

    # La réservation échoue
    with pytest.raises(ValueError):
        reserver_stock(product_sample, 999, db_session)

    # Le stock ne doit PAS avoir changé
    assert product_sample.stock == stock_avant
```

```python
    def test_redis_non_appele_si_exception(self, product_sample, db_session, mocker):
        """
        Si la réservation échoue (stock insuffisant),
        Redis.delete() ne doit PAS être appelé.
        Aucune raison d'invalider le cache si rien n'a changé.
        """
        mock_redis = mocker.patch(REDIS_MOCK_PATH)

        with pytest.raises(ValueError):
            reserver_stock(product_sample, 999, db_session)

        # delete() ne doit pas avoir été appelé
        mock_redis.delete.assert_not_called()



# TESTS liberer_stock()
#
# Cette fonction fait 3 choses :
#   1. Vérifie que la quantité est valide (> 0)
#   2. Incrémente product.stock en BDD (session.commit)
#   3. Met à jour le cache Redis (redis_client.set)
#
# Cas d'usage : annulation d'une commande → stock restitué


    def test_liberation_stock(self, product_sample, db_session, mocker):
        """
        Cas nominal : libérer 2 unités → stock augmente de 2.

        Étapes du test :
            1. Mocker Redis
            2. Appeler liberer_stock()
            3. Vérifier que le stock a augmenté de 2
            4. Vérifier que Redis.set() a bien été appelé
               (le cache doit être mis à jour avec le nouveau stock)
        """
       # Completer le code
```

Règle fondamentale du mocking

- Mocker uniquement aux frontières I/O : Redis, APIs externes, fichiers, emails, jamais la logique interne.
- mocker.patch('app.cache.redis_client') remplace l'objet redis_client dans le module stock.py par un MagicMock.
- Le mock est automatiquement supprimé après chaque test, pas besoin de cleanup manuel.
- assert_called_once() vérifie que la fonction mockée a été appelée exactement une fois, essentiel pour tester les effets de bord.

Q3 : Tests Module Stock & Mocking

3.1. Implémentez tous les tests de TestVerifierStock et TestReserverStock. Lancez pytest tests/unit/test_stock.py -v et copiez la sortie.

3.2. Ajoutez le test test_liberation_stock() et vérifiez que mock_redis.set() est appelé avec les bons arguments (key et valeur).

3.3. Que se passe-t-il si vous supprimez mocker.patch('app.cache.redis_client') du test test_reservation_reussie() ? Expliquez l'erreur obtenue.

3.4. Utilisez mocker.patch.object() au lieu de mocker.patch() sur un des tests. Expliquez la différence entre les deux approches.

Partie 4 : Coverage et Rapport

4.1 Lancer la suite complète avec coverage

```
# Lancer tous les tests unitaires avec rapport coverage
pytest tests/unit/ -v --cov=app --cov-report=html --cov-report=term-missing

# Exemple de sortie attendue :
# tests/unit/test_pricing.py::TestCalculPrixTtc::test_prix_normal PASSED
# tests/unit/test_pricing.py::TestCalculPrixTtc::test_prix_zero PASSED
# ...
# ---- coverage: platform linux ----
# Name                          Stmts   Miss  Cover
# -----------------------------------------------
# app/services/pricing.py          18      0   100%
# app/services/stock.py            14      2    86%
# app/services/cart.py             22      5    77%   ← à améliorer !
# TOTAL                            54      7    87%
```

4.2 Interpréter le rapport HTML

Le rapport HTML est généré dans `htmlcov/index.html`. Ouvrez-le dans un navigateur pour identifier les lignes non couvertes (affichées en rouge).

Lecture du rapport Coverage.py

- Vert (covered) : la ligne a été exécutée par au moins un test.
- Rouge (missing) : la ligne n'a jamais été exécutée — il manque un test pour ce chemin de code.
- Jaune (partial) : la condition n'a été testée que dans un sens (ex: if True mais jamais if False).
- Objectif : 0 lignes rouges dans les modules services/ et routes/.
- Exclure du rapport : les fichiers de migration, conftest.py, et __init__.py via .coveragerc.

4.3 Atteindre 80% de coverage

Si votre coverage est inférieur à 80%, identifiez les lignes non couvertes et écrivez les tests manquants. Voici les cas typiques non testés :

```
# Cas souvent oubliés — à couvrir obligatoirement :

# 1. Les branches else des conditions
#    if not produits: return 0.0  → tester avec liste vide
def test_calculer_total_liste_vide():
    assert calculer_total([]) == 0.0

# 2. Les exceptions levées
#    raise ValueError(...)  → tester avec pytest.raises
def test_prix_negatif():
    with pytest.raises(ValueError):
        calcul_prix_ttc(-1.0)
```

```
# 3. Les appels de fonctions dans les branches
#    redis_client.set(...)  → vérifier avec assert_called
def test_liberer_appelle_redis(mocker, product_sample, db_session):
    mock = mocker.patch('app.cache.redis_client')
    liberer_stock(product_sample, 1, db_session)
    mock.set.assert_called_once()
```

Q4 : Coverage & Analyse

4.1. Lancez la suite complète et copiez la sortie du rapport coverage (section terminal). Indiquez votre taux de coverage total.

4.2. Ouvrez htmlcov/index.html. Identifiez 3 lignes non couvertes dans le code source et expliquez pourquoi elles ne l'étaient pas.

4.3. Écrivez les tests manquants pour atteindre ≥ 80% de coverage sur l'ensemble de app/services/. Montrez la nouvelle sortie coverage.

4.4. Ajoutez fail_under = 80 dans .coveragerc. Que se passe-t-il si vous lancez pytest avec un coverage de 75% ?